

ÉCOLE MOHAMMADIA D'INGÉNIEURS

Département Informatique & Digitalisation

FONDEMENTS DE LA CYBERSÉCURITÉ Projet N9

UFW Uncomplicated Firewall

Attaque, Détection & Protection par Pare-feu

Ce rapport présente une étude complète du pare-feu UFW sous Ubuntu, selon une démarche d'ingénieur sécurité. Nous simulons trois attaques réalistes depuis Kali Linux (force brute SSH, serveur Python vulnérable, attaque DVWA), puis analysons les traces système. Enfin, nous déployons des règles UFW adaptées en soignant l'ordre des règles pour une politique de sécurité robuste.

Réalisé par :

ABOUBAKAR Abdelaziz
DIN Isaac Kaougahi
KAFANDO Mohamed
ZOUNGRANA Abdoul Gafarou

Sous la direction de :

Pr. Asmae KASSIRI

Résumé exécutif

Ce rapport a été réalisé dans le cadre du module **Sécurité des Réseaux & Systèmes** de l'École Mohammadia d'Ingénieurs. Il porte sur la prise en main et la maîtrise du pare-feu **UFW (Uncomplicated Firewall)**, un outil de filtrage de trafic réseau intégré à Linux et permettant de gérer simplement les règles `iptables`.

Pourquoi ce projet commence par des attaques ?

La compétence d'un ingénieur en sécurité ne se résume pas à savoir configurer un outil. Elle réside dans la capacité à *comprendre l'attaquant*, à interpréter les traces qu'il laisse, et à concevoir des contre-mesures adaptées. C'est pourquoi ce rapport adopte une structure en deux temps :

Phase 1. Attaques — Nous simulons trois types d'attaques réalistes dans un environnement virtualisé isolé (Kali Linux contre Ubuntu), en documentant précisément les outils, les commandes, et les résultats obtenus.

Phase 2. Défense — Pour chaque attaque, nous déployons des règles UFW ciblées, nous vérifions leur efficacité, et nous analysons l'importance de l'ordre des règles.

Les trois attaques simulées sont :

- **Attaque 1 — Force brute sur SSH** : Utilisation de Hydra après un scan Nmap pour découvrir les identifiants du service SSH ouvert sur la machine cible. Cette attaque illustre le risque d'un port d'administration exposé sans protection.
- **Attaque 2 — Serveur HTTP Python exposé** : Simulation d'une erreur de configuration courante : un développeur lance un serveur Python sur le port 8080 sans le refermer. L'attaquant accède alors librement aux fichiers servis, y compris des données sensibles.
- **Attaque 3 — Attaque sur DVWA (application web vulnérable)** : Installation et exploitation de l'application DVWA (*Damn Vulnerable Web Application*) pour simuler une attaque par dictionnaire sur un formulaire de connexion web, illustrant la vulnérabilité des services applicatifs.

Ce que vous trouverez dans ce rapport :

- Comment configurer un environnement de test virtualisé sécurisé.

- Comment utiliser Nmap, Hydra et d'autres outils offensifs dans un cadre éthique.
- Comment lire et interpréter les journaux système Linux (`auth.log`, `ufw.log`).
- Comment configurer UFW avec les commandes `allow`, `deny`, `limit`, et `insert`.
- Pourquoi l'ordre des règles dans un pare-feu est critique.
- Comment établir une politique de sécurité finale structurée.

Contents

Résumé exécutif	1
1 Introduction générale	6
1.1 Contexte et motivation	6
1.2 Qu'est-ce qu'UFW ?	6
1.3 Objectifs pédagogiques	7
1.4 Organisation du rapport	7
2 Environnement de test et configuration initiale	9
2.1 Architecture générale de l'environnement	9
2.2 Configuration de la machine cible (Ubuntu 22.04)	10
2.2.1 Mise à jour du système	10
2.2.2 Installation et activation du serveur SSH	10
2.2.3 Installation des prérequis pour le serveur web (Apache, PHP, MySQL)	11
2.2.4 Création de l'utilisateur de test	12
2.3 Configuration de la machine attaquante (Kali Linux)	13
2.3.1 Vérification des outils disponibles	13
2.3.2 Création du dictionnaire de mots de passe	13
2.3.3 Test de connectivité réseau	13
3 Reconnaissance préalable avec Nmap	15
3.1 Pourquoi la reconnaissance ?	15
3.2 Scan des ports ouverts	15
3.3 Analyse des résultats	16
3.4 Informations complémentaires récupérées par les scripts NSE	17
4 Attaque n1 : Force brute sur le service SSH	18
4.1 Concept de l'attaque par force brute	18

4.2	Présentation de l'outil Hydra	18
4.3	Exécution de l'attaque (sans protection UFW)	19
4.4	Analyse des journaux système sur la machine cible	20
5	Attaque n2 : Exposition accidentelle d'un serveur HTTP Python	21
5.1	Contexte et scénario	21
5.2	Préparation du serveur HTTP sur Ubuntu	21
5.3	Vérification avec <code>ufw status</code>	22
5.4	Exécution de l'attaque depuis Kali Linux	22
5.5	Observation côté serveur	22
6	Attaque n3 : Application web vulnérable DVWA	24
6.1	Exécution de l'attaque par force brute sur le formulaire DVWA	24
6.1.1	Analyse du formulaire de connexion	24
6.1.2	Accès au formulaire	26
7	Déploiement d'UFW — Contre-mesures	27
7.1	Introduction à UFW	27
7.1.1	Architecture interne d'UFW	27
7.1.2	Commandes fondamentales	28
7.2	Protection contre l'attaque SSH (force brute)	29
7.2.1	Activation d'UFW et règle <code>limit</code>	29
7.2.2	Relance de l'attaque Hydra après protection	29
7.2.3	Bannissement manuel de l'IP attaquante	30
7.3	Protection contre l'exposition HTTP Python (port 8080)	30
7.3.1	Application de la règle <code>deny</code>	30
7.3.2	Vérification de l'efficacité	31
7.4	Protection contre les attaques web (DVWA, port 80)	31
8	Analyse critique de l'ordre des règles UFW	33
8.1	Le principe "First Match Wins"	33

8.2	Scénario problématique : blocage inopérant	33
8.3	Correction : Insérer le blocage en première position	34
8.4	Tableau récapitulatif de l'impact de l'ordre	35
9	Politique de sécurité finale	36
9.1	Principes directeurs	36
9.2	Configuration complète et commentée	37
9.3	Récapitulatif de la politique	38
9.4	Surveillance continue des journaux UFW	38
10	Conclusion	40
10.1	Bilan du projet	40
10.2	Perspectives	40
A	Glossaire	42

Chapter 1

Introduction générale

1.1 Contexte et motivation

Dans un système d'information, la sécurité périmétrique constitue la première ligne de défense. Parmi les mécanismes disponibles, le **pare-feu** (*firewall*) joue un rôle fondamental : il filtre le trafic entrant et sortant selon un ensemble de règles, permettant de n'autoriser que les communications légitimes et de bloquer toutes les autres.

Sous Linux, plusieurs implémentations de pare-feu coexistent : **iptables**, **nftables**, **firewalld**, et **UFW (Uncomplicated Firewall)**. Ce dernier se distingue par son interface simplifiée tout en s'appuyant sur la puissance d'**iptables** en arrière-plan. Il est particulièrement adapté aux serveurs Ubuntu et aux environnements de production de taille moyenne.

L'enjeu de ce projet est de dépasser la simple documentation des commandes UFW pour acquérir une **compréhension mécanique** de son fonctionnement : comment les règles sont-elles évaluées ? Dans quel ordre ? Quelles sont les règles adaptées à chaque type de menace ? C'est cette profondeur de compréhension que ce projet cherche à démontrer.

1.2 Qu'est-ce qu'UFW ?

Definition

UFW — Uncomplicated Firewall est un frontal (*frontend*) à la commande **iptables** conçu pour simplifier la gestion des règles de filtrage sous Linux. Développé par Canonical pour Ubuntu, il permet de gérer les règles de filtrage par des commandes intuitives telles que `sudo ufw allow 22/tcp`, sans manipuler directement les chaînes **iptables**.

UFW fonctionne selon un modèle de filtrage stateless par défaut (chaque paquet est évalué indépendamment), mais peut intégrer un suivi de connexion (*stateful*) via les modules `conntrack` d’iptables. Les règles sont évaluées dans l’ordre dans lequel elles apparaissent dans la table : la première règle qui correspond au paquet est appliquée, et les suivantes ne sont pas consultées. Ce principe dit “**first match wins**” est au cœur de ce projet.

1.3 Objectifs pédagogiques

Ce projet poursuit les objectifs suivants :

1. Mettre en place un environnement virtualisé isolé et réaliste.
2. Comprendre et utiliser les outils de reconnaissance réseau (Nmap).
3. Simuler des attaques réalistes (force brute, exposition accidentelle, attaque web).
4. Analyser les traces laissées dans les journaux système.
5. Configurer UFW de manière progressive et raisonnée pour contrer chaque menace.
6. Comprendre l’impact de l’ordre des règles UFW.
7. Établir une politique de sécurité finale complète et documentée.

1.4 Organisation du rapport

Le rapport est organisé de la manière suivante :

- **Chapitre 2** : Description de l’architecture virtualisée et configuration initiale des deux machines.
- **Chapitre 3** : Reconnaissance préalable avec Nmap.
- **Chapitre 4** : Attaque n1 — Force brute SSH avec Hydra.
- **Chapitre 5** : Attaque n2 — Serveur HTTP Python exposé.
- **Chapitre 6** : Attaque n3 — Application web DVWA.

- **Chapitre 7** : Déploiement d'UFW et contre-mesures pour chaque attaque.
- **Chapitre 8** : Analyse approfondie de l'ordre des règles.
- **Chapitre 9** : Politique de sécurité finale.
- **Chapitre 10** : Conclusion.

Chapter 2

Environnement de test et configuration initiale

2.1 Architecture générale de l'environnement

Pour garantir la sécurité et l'isolation des expériences, nous avons mis en place un environnement entièrement virtualisé composé de deux machines virtuelles communiquant exclusivement au sein d'un réseau local interne (sans accès à Internet), configuré sous Oracle VirtualBox en mode **Interne Network** (ou **Host-Only Network**).

Cette isolation est fondamentale : elle garantit que les attaques simulées ne peuvent pas s'échapper vers un réseau externe réel, et que l'environnement reproduit fidèlement les conditions d'un réseau d'entreprise cloisonné.

Table 2.1: Configuration des machines virtuelles

Rôle	Système d'exploitation	Adresse IP	Outils installés
Machine cible	Ubuntu 22.04 LTS	10.72.153.175	OpenSSH Server, Python3, Apache2, MySQL, PHP, DVWA
Machine attaquante	Kali Linux 2024.3	10.72.178.175	Nmap, Hydra, curl, navigateur Firefox

Information

Le choix de ces deux systèmes est standard dans le domaine de la sécurité offensive : **Kali Linux** est la distribution de référence pour les tests de pénétration (elle intègre nativement Nmap, Hydra, Metasploit, etc.) et **Ubuntu** représente un serveur Linux standard en production.

Schéma de l'architecture réseau

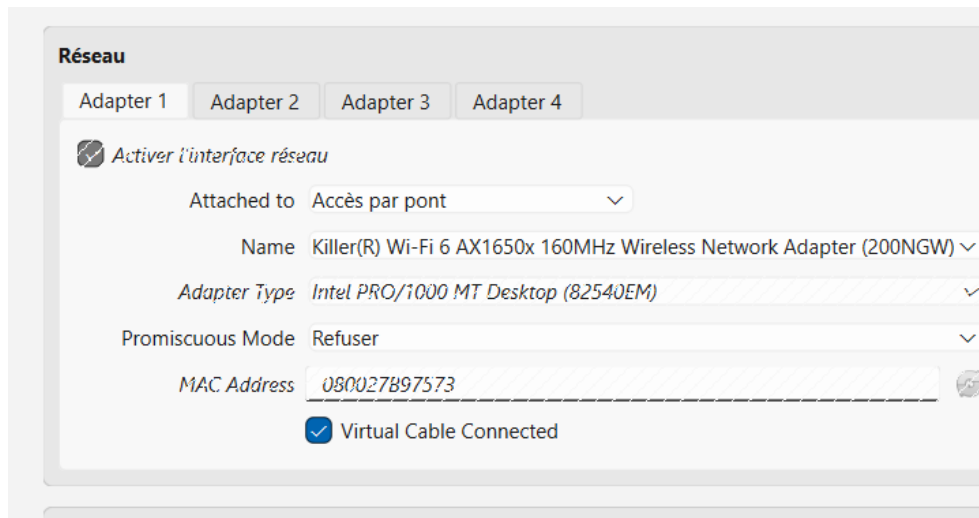


Figure 2.1: $\text{accès}_{\text{par}_{\text{pont}}}$

2.2 Configuration de la machine cible (Ubuntu 22.04)

2.2.1 Mise à jour du système

Avant toute installation, il est impératif de s'assurer que les paquets du système sont à jour afin d'éliminer les vulnérabilités connues des versions obsolètes :

```
1 sudo apt update && sudo apt upgrade -y
```

Listing 2.1: Mise à jour du système Ubuntu

2.2.2 Installation et activation du serveur SSH

Le service SSH (*Secure Shell*) est le premier service que nous allons exposer et attaquer. Il permet l'administration distante de la machine via un terminal chiffré. OpenSSH est le serveur SSH standard sous Linux.

```
1 sudo apt install openssh-server -y
2 sudo systemctl enable ssh
3 sudo systemctl start ssh
4 sudo systemctl status ssh
```

Listing 2.2: Installation et démarrage du serveur OpenSSH

Après exécution, la commande `systemctl status ssh` doit afficher `active (running)`. Le port 22/TCP est maintenant écouté sur toutes les interfaces réseau de la machine.

```
abdoulgafarouzoungana@Ubuntu:~$ sudo ufw status
Status: inactive
abdoulgafarouzoungana@Ubuntu:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enab>
   Active: active (running) since Fri 2026-04-03 14:23:55 UTC; 13min ago
   TriggeredBy: ● ssh.socket
     Docs: man:sshd(8)
           man:sshd_config(5)
   Process: 1398 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 1424 (sshd)
     Tasks: 1 (limit: 4599)
    Memory: 2.1M (peak: 2.5M)
       CPU: 24ms
    CGroup: /system.slice/ssh.service
           └─1424 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Apr 03 14:23:55 Ubuntu systemd[1]: Starting ssh.service - OpenBSD Secure Shell >
Apr 03 14:23:55 Ubuntu sshd[1424]: Server listening on 0.0.0.0 port 22.
Apr 03 14:23:55 Ubuntu sshd[1424]: Server listening on :: port 22.
Apr 03 14:23:55 Ubuntu systemd[1]: Started ssh.service - OpenBSD Secure Shell s>
lines 1-18/18 (END)
```

Figure 2.2: ufw satuts et serve ssh

Attention

À ce stade, le pare-feu UFW est **inactif par défaut** sous Ubuntu (`sudo ufw status` retourne `Status: inactive`). Cela signifie que le port 22 est accessible depuis n'importe quelle machine du réseau, sans aucun contrôle d'accès. C'est précisément cette situation que nous allons exploiter dans l'attaque n1.

2.2.3 Installation des prérequis pour le serveur web (Apache, PHP, MySQL)

La troisième attaque portera sur l'application DVWA, qui nécessite une pile web complète. Nous installons la pile LAMP (**L**inux, **A**pache, **M**ySQL, **P**HP) :

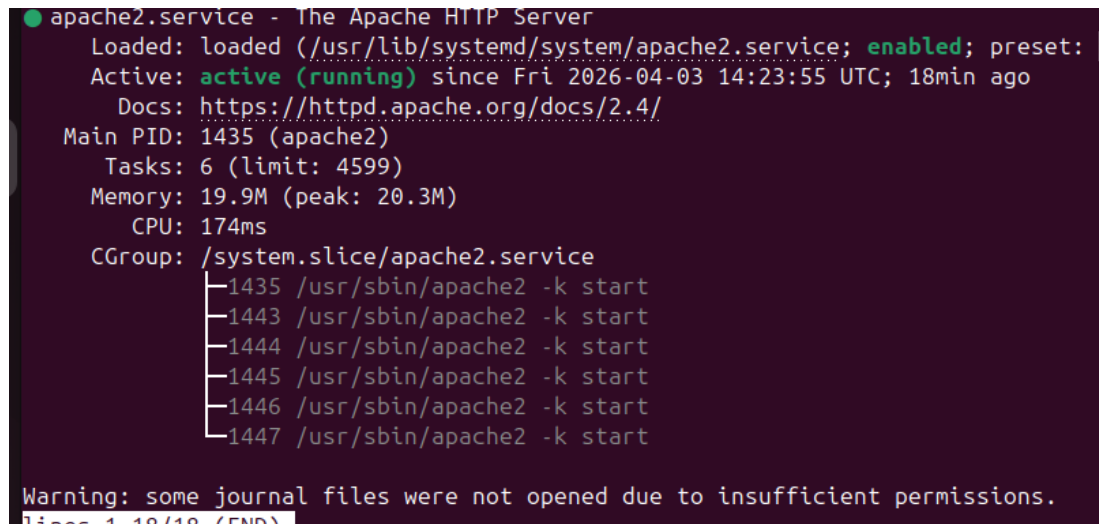
```
1 sudo apt install apache2 mysql-server php php-mysqli php-gd \
2   php-xml libapache2-mod-php -y
3 sudo systemctl enable apache2
4 sudo systemctl enable mysql
5 sudo systemctl start apache2
6 sudo systemctl start mysql
```

Listing 2.3: Installation de la pile LAMP

Apache écoute sur le port 80/TCP par défaut. Nous pouvons vérifier que le service fonctionne avec :

```
1 sudo systemctl status apache2
2 sudo systemctl status mysql
```

Listing 2.4: Vérification des services web



```
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset:
   Active: active (running) since Fri 2026-04-03 14:23:55 UTC; 18min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 1435 (apache2)
    Tasks: 6 (limit: 4599)
   Memory: 19.9M (peak: 20.3M)
      CPU: 174ms
   CGroup: /system.slice/apache2.service
           └─1435 /usr/sbin/apache2 -k start
             └─1443 /usr/sbin/apache2 -k start
               └─1444 /usr/sbin/apache2 -k start
                 └─1445 /usr/sbin/apache2 -k start
                   └─1446 /usr/sbin/apache2 -k start
                     └─1447 /usr/sbin/apache2 -k start

Warning: some journal files were not opened due to insufficient permissions.
lines 1-18/18 (END)
```

Figure 2.3: Résultat de `systemctl status apache2`

2.2.4 Création de l'utilisateur de test

L'utilisateur `abdoulgafarouzoungrana` est créé avec le mot de passe volontairement faible `1234` pour illustrer les risques d'une politique de mots de passe insuffisante :

```
1 sudo adduser abdoulgafarouzoungrana
2 # Mot de passe : 1234 (volontairement faible pour la
   demonstration)
```

Listing 2.5: Création de l'utilisateur cible

Point critique

Utiliser un mot de passe comme `1234` est une **faute de sécurité grave** en production. Ce choix est ici intentionnel et pédagogique : il permet de montrer qu'un attaquant avec un simple dictionnaire peut compromettre un compte en quelques secondes.

2.3 Configuration de la machine attaquante (Kali Linux)

2.3.1 Vérification des outils disponibles

Kali Linux intègre nativement les outils nécessaires à nos tests. Nous vérifions leur disponibilité :

```
1 which nmap
2 which hydra
3 which curl
4 nmap --version
5 hydra -h | head -5
```

Listing 2.6: Vérification des outils sur Kali Linux

2.3.2 Création du dictionnaire de mots de passe

Plutôt que d'utiliser le fichier `rockyou.txt` (contenant plus de 14 millions d'entrées), nous créons un mini-dictionnaire ciblé contenant le mot de passe réel parmi quelques décoys. Cela permet de contrôler la durée de la démonstration :

```
1 echo -e "admin\npassword\n123456\n1234\ntoto\nroot\nqwerty" >
  mini_dico.txt
2 cat mini_dico.txt
```

Listing 2.7: Création du dictionnaire sur Kali Linux

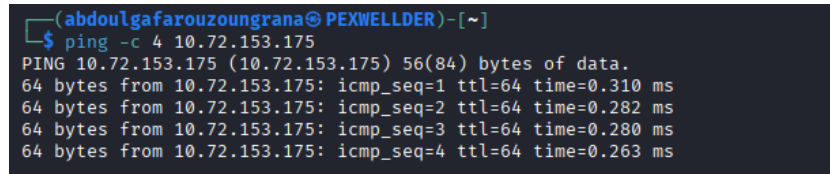
Le fichier `mini_dico.txt` contient sept entrées. Hydra les essaiera dans l'ordre, et trouvera 1234 en quatrième position.

2.3.3 Test de connectivité réseau

Avant de lancer toute attaque, nous vérifions que la machine cible est accessible :

```
1 ping -c 4 10.72.153.175
```

Listing 2.8: Test de connectivité depuis Kali vers Ubuntu

A terminal window with a dark background. The prompt is `(abdoulgafarouzungrana@PEXWELLDER)~`. The user enters `$ ping -c 4 10.72.153.175`. The output shows four successful ping responses from 10.72.153.175 with varying times.

```
(abdoulgafarouzungrana@PEXWELLDER)~  
$ ping -c 4 10.72.153.175  
PING 10.72.153.175 (10.72.153.175) 56(84) bytes of data.  
64 bytes from 10.72.153.175: icmp_seq=1 ttl=64 time=0.310 ms  
64 bytes from 10.72.153.175: icmp_seq=2 ttl=64 time=0.282 ms  
64 bytes from 10.72.153.175: icmp_seq=3 ttl=64 time=0.280 ms  
64 bytes from 10.72.153.175: icmp_seq=4 ttl=64 time=0.263 ms
```

Figure 2.4: ping

Chapter 3

Reconnaissance préalable avec Nmap

3.1 Pourquoi la reconnaissance ?

Avant de lancer une attaque, tout attaquant — comme tout auditeur de sécurité — commence par une phase de **reconnaissance** (*reconnaissance* ou *enumeration*). L'objectif est de cartographier la surface d'attaque : quels ports sont ouverts ? Quels services y tournent ? Quelles versions ? Ces informations guident les choix des outils et techniques d'attaque.

Definition

Nmap (*Network Mapper*) est l'outil de référence pour la reconnaissance réseau. Il envoie des paquets soigneusement conçus aux hôtes cibles et analyse leurs réponses pour déterminer : quels hôtes sont actifs, quels ports sont ouverts, quels services écoutent sur ces ports, et quelle version du service est en cours d'exécution.

3.2 Scan des ports ouverts

Nous lançons un scan complet avec détection de version sur tous les ports TCP :

```
1 nmap -sV -sC -p- 10.72.153.175
```

Listing 3.1: Scan Nmap complet sur la machine cible

Explication des options :

- **-sV** : Détection de version des services (*service version detection*). Nmap envoie des sondes spécifiques pour identifier le logiciel et sa version.
- **-sC** : Exécute les scripts NSE (*Nmap Scripting Engine*) par défaut. Ces scripts récupèrent des informations supplémentaires (bannières, algorithmes de chiffrement

SSH, etc.).

- `-p-` : Scanne l'intégralité des 65 535 ports TCP (par défaut Nmap ne scanne que les 1000 ports les plus communs).

```
(abdoulgafarouzungrana@PEXWELLDER)~$ nmap -sV -sC -p- 10.72.153.175
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-03 16:04 +01
Nmap scan report for 10.72.153.175
Host is up (0.00022s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 256 94:cc:aa:3f:42:73:0e:c4:23:82:07:83:5f:a0:34:e4 (ECDSA)
|_ 256 57:d6:b1:7a:30:ed:ed:7e:90:12:5b:1b:bc:34:65:85 (ED25519)
80/tcp    open  http      Apache httpd 2.4.58 ((Ubuntu))
|_ http-server-header: Apache/2.4.58 (Ubuntu)
|_ http-title: Apache2 Ubuntu Default Page: It works
8080/tcp  open  http      SimpleHTTPServer 0.6 (Python 3.12.3)
|_ http-title: Site doesn't have a title (text/html).
|_ http-server-header: SimpleHTTP/0.6 Python/3.12.3
MAC Address: 08:00:27:B9:75:73 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.61 seconds
```

Figure 3.1: Sortie complète du scan Nmap montrant les ports 22, 80 et 8080 ouverts avec les versions des services

3.3 Analyse des résultats

Le scan révèle trois ports ouverts, qui constitueront nos trois vecteurs d'attaque :

Table 3.1: Ports et services découverts par Nmap

Port	Protocole	Service détecté	Attaque associée
22/TCP	SSH	OpenSSH 8.9p1 Ubuntu	Force brute (Hydra)
80/TCP	HTTP	Apache httpd 2.4.52	Attaque DVWA
8080/TCP	HTTP	SimpleHTTP/0.6 Python 3.10.12	Accès non autorisé

Point critique

Un scan Nmap est l'équivalent numérique d'un cambrioleur qui tourne autour d'un bâtiment et vérifie toutes les portes et fenêtres. Le fait que ces informations soient visibles depuis le réseau suffit à permettre un ciblage précis des attaques. La mise en place d'UFW, que nous verrons au chapitre 7, permettra de réduire drastiquement ces informations visibles.

3.4 Informations complémentaires récupérées par les scripts NSE

Les scripts Nmap par défaut (`-sC`) nous fournissent des informations supplémentaires utiles sur le service SSH :

```
1 22/tcp open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.10
2 | ssh-hostkey:
3 |   256 xx:xx:xx:... (ECDSA)
4 |_  256 xx:xx:xx:... (ED25519)
```

Listing 3.2: Extrait de la sortie NSE pour le service SSH

Ces informations (version exacte d'OpenSSH, algorithmes de clés supportés) permettent à un attaquant de rechercher les CVE (*Common Vulnerabilities and Exposures*) associées à cette version spécifique.

Chapter 4

Attaque n1 : Force brute sur le service SSH

4.1 Concept de l'attaque par force brute

Definition

Une **attaque par force brute** (*brute force attack*) consiste à tester systématiquement toutes les combinaisons possibles de mots de passe jusqu'à trouver la bonne. Une variante plus efficace est l'**attaque par dictionnaire** (*dictionary attack*), qui utilise une liste de mots de passe prédéfinis plutôt que toutes les combinaisons possibles.

Le service SSH utilise un système d'authentification par nom d'utilisateur et mot de passe. Sans limitation du nombre de tentatives, un attaquant peut essayer autant de mots de passe qu'il le souhaite, limité uniquement par la bande passante réseau et la puissance de calcul disponible.

4.2 Présentation de l'outil Hydra

Definition

Hydra (ou *THC-Hydra*) est un outil de force brute réseau parallélisé. Il supporte plus de 50 protocoles (SSH, FTP, HTTP, RDP, SMTP...) et peut lancer plusieurs connexions simultanées pour accélérer l'attaque. Il est inclus par défaut dans Kali Linux.

La syntaxe générale d'Hydra est la suivante :

```
1 hydra -l <utilisateur> -P <fichier_dico> <protocole>://<cible> [  
    options]
```

Les options principales utilisées dans ce projet :

- `-l` : Spécifie un nom d'utilisateur unique (minuscule L).
- `-P` : Spécifie le fichier dictionnaire de mots de passe (majuscule P).
- `-t 4` : Limite à 4 connexions simultanées (respecte la cible).
- `-V` : Mode verbeux, affiche chaque tentative en temps réel.

4.3 Exécution de l'attaque (sans protection UFW)

Depuis la machine Kali Linux, nous lançons l'attaque par dictionnaire sur le service SSH de la machine cible :

```
1 hydra -l abdoulgafarouzoungrana -P mini_dico.txt \  
2 ssh://10.72.153.175 -t 4 -V
```

Listing 4.1: Commande Hydra pour l'attaque SSH — Machine Kali

Détail de ce qui se passe : Hydra va successivement essayer les combinaisons suivantes sur le port 22 de la cible :

1. `abdoulgafarouzoungrana:admin` → échec
2. `abdoulgafarouzoungrana:password` → échec
3. `abdoulgafarouzoungrana:123456` → échec
4. `abdoulgafarouzoungrana:1234` → **succès !**

```
(abdoulgafarouzoungana@PEXWELLDER)~$ hydra -l abdoulgafarouzoungana -P mini_dico.txt \
ssh://10.72.153.175 -t 4 -V
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizati
ons, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-03 16:11:19
[DATA] max 4 tasks per 1 server, overall 4 tasks, 8 login tries (l:1/p:8), ~2 tries per task
[DATA] attacking ssh://10.72.153.175:22/
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "password" - 1 of 8 [child 0] (0/0)
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "admin" - 2 of 8 [child 1] (0/0)
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "password" - 3 of 8 [child 2] (0/0)
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "123456" - 4 of 8 [child 3] (0/0)
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "45666" - 5 of 8 [child 0] (0/0)
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "5446313" - 6 of 8 [child 1] (0/0)
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "656532" - 7 of 8 [child 2] (0/0)
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "1234" - 8 of 8 [child 3] (0/0)
[22][ssh] host: 10.72.153.175 login: abdoulgafarouzoungana password: 1234
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-03 16:11:23
```

Figure 4.1: Sortie complète d’Hydra montrant les tentatives en mode verbeux (-V) et la ligne de succès : [22][ssh] host: 10.72.153.175 login: abdoulgafarouzoungana password: 1234

Point critique

L’attaque a réussi en quelques secondes. Un vrai attaquant avec un dictionnaire comme `rockyou.txt` (14 millions d’entrées) aurait trouvé ce mot de passe en moins d’une heure, même sans accélérateur matériel.

4.4 Analyse des journaux système sur la machine cible

Chaque tentative de connexion SSH génère une entrée dans le journal `/var/log/auth.log`. Nous l’observons en temps réel :

```
1 sudo tail -f /var/log/auth.log
```

Listing 4.2: Consultation des journaux d’authentification — Machine Ubuntu

La sortie révèle une série caractéristique de tentatives échouées (`Failed password`) suivie d’une connexion réussie (`Accepted password`) :

```
a
2026-04-03T15:20:10.263823+00:00 localhost sshd[4619]: Failed password for abdoulgafarouzoungana from
2026-04-03T15:20:10.264376+00:00 localhost sshd[4617]: Failed password for abdoulgafarouzoungana from
2026-04-03T15:20:10.264667+00:00 localhost sshd[4618]: Failed password for abdoulgafarouzoungana from
2026-04-03T15:20:10.264983+00:00 localhost sshd[4620]: Failed password for abdoulgafarouzoungana from
2026-04-03T15:20:12.132380+00:00 localhost sshd[4620]: Accepted password for abdoulgafarouzoungana from
2026-04-03T15:20:12.135418+00:00 localhost sshd[4620]: pam_unix(sshd:session): session opened for use
```

Figure 4.2: Capture de la console Ubuntu montrant le flux de `auth.log` en temps réel avec les lignes `Failed password` et la ligne `Accepted password` finale

Chapter 5

Attaque n2 : Exposition accidentelle d'un serveur HTTP Python

5.1 Contexte et scénario

Definition

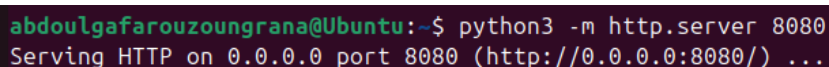
Scénario typique : Un développeur ou un administrateur, sur la machine Ubuntu, souhaite transférer rapidement un fichier. Pour cela, il utilise la commande pratique `python3 -m http.server`, qui lance en quelques secondes un serveur HTTP minimaliste. Ce faisant, il oublie souvent que ce serveur expose **l'intégralité de l'arborescence** du répertoire courant à quiconque peut atteindre le port réseau concerné.

Ce scénario est **extrêmement courant** dans les environnements de développement et de test. Il illustre une vulnérabilité organisationnelle plutôt que technique : l'absence de politique claire sur l'utilisation des services réseau temporaires.

5.2 Préparation du serveur HTTP sur Ubuntu

lançons le serveur Python :

Le serveur Python est maintenant actif sur le port 8080 et expose le contenu de la machine ubuntu



```
abdoulgafarouzoungwana@Ubuntu:~$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Figure 5.1: Lancement du serveur

5.3 Vérification avec `ufw status`

```
abdoulgafarouzougrana@Ubuntu:~$ sudo ufw status
[sudo] password for abdoulgafarouzougrana:
Status: inactive
abdoulgafarouzougrana@Ubuntu:~$
```

Figure 5.2: Verifivation du status de ufw

Avant l'attaque, nous vérifions l'état d'UFW sur la machine cible pour confirmer l'absence de protection :

5.4 Exécution de l'attaque depuis Kali Linux

Depuis la machine attaquante, nous accédons au serveur HTTP Python de deux manières : via le terminal et nous arrivons meme a telecharger des fichiers de la machine UBUNTU A PARTIR DE KALI:

```
(abdoulgafarouzougrana@PEXWELLDER)-[~]
$ wget http://10.72.153.175:8080/ex3
--2026-04-03 16:32:39-- http://10.72.153.175:8080/ex3
Connexion à 10.72.153.175:8080... connecté.
requête HTTP transmise, en attente de la réponse... 200 OK
Taille : 568 [application/octet-stream]
Sauvegarde en : « ex3.2 »

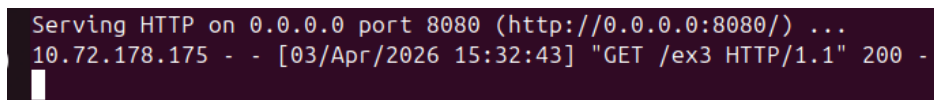
ex3.2                               100%[=====>]          568  --.-KB/s   ds 0s
2026-04-03 16:32:39 (3,86 MB/s) - « ex3.2 » sauvegardé [568/568]

(abdoulgafarouzougrana@PEXWELLDER)-[~]
$
```

Figure 5.3: Telechargement de ex , un fichier de la machine ubuntu

5.5 Observation côté serveur

Sur la machine Ubuntu, le serveur Python affiche les accès en temps réel : on voit bien que le fichier ex 3 a ete telechargé

A terminal window with a dark background and light-colored text. The text shows an HTTP server running on 0.0.0.0 port 8080. It logs a GET request from 10.72.178.175 at 15:32:43 on 03/Apr/2026, returning a 200 status code. A cursor is visible at the end of the second line.

```
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...  
10.72.178.175 - - [03/Apr/2026 15:32:43] "GET /ex3 HTTP/1.1" 200 -  
█
```

Figure 5.4: affichage du serveur sur ubuntu

Chapter 6

Attaque n3 : Application web vulnérable DVWA

6.1 Exécution de l'attaque par force brute sur le formulaire DVWA

6.1.1 Analyse du formulaire de connexion

Sur ubuntu , Nous installons d'abord un environnement LAMP (Apache, MariaDB, PHP) sur notre machine Ubuntu. Nous déplaçons ensuite les sources dans le répertoire /var/www/html/ avant de configurer nos identifiants SQL dans le fichier config.inc.php. Pour finir, nous validons la création de la base de données via notre navigateur pour générer les tables et afficher enfin notre formulaire de connexion.

```
#!/php

If you are having problems connecting to the MySQL database and all of the variables below are correct
try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a problem due to sockets.
Thanks to @digininja for the fix.

Database management system to use
DBMS = getenv('DBMS') ?: 'MySQL';
$DBMS = 'PGSQL'; // Currently disabled

Database variables
WARNING: The database specified under db_database WILL BE ENTIRELY DELETED during setup.
Please use a database dedicated to DVWA.

If you are using MariaDB then you cannot use root, you must use create a dedicated DVWA user.
See README.md for more information on this.
_DVWA = array();
_DVWA[ 'db_server' ] = getenv('DB_SERVER') ?: '127.0.0.1';
_DVWA[ 'db_database' ] = getenv('DB_DATABASE') ?: 'dvwa';
_DVWA[ 'db_user' ] = getenv('DB_USER') ?: 'dvwa';
_DVWA[ 'db_password' ] = '';
_DVWA[ 'db_port' ] = getenv('DB_PORT') ?: '3306';

ReCAPTCHA settings
Used for the 'Insecure CAPTCHA' module
You'll need to generate your own keys at: https://www.google.com/recaptcha/admin
_DVWA[ 'recaptcha_public_key' ] = getenv('RECAPTCHA_PUBLIC_KEY') ?: '';
_DVWA[ 'recaptcha_private_key' ] = getenv('RECAPTCHA_PRIVATE_KEY') ?: '';

Default security level
Default value for the security level with each session.
The default is 'impossible'. You may wish to set this to either 'low', 'medium', 'high' or 'impossible'.
_DVWA[ 'default_security_level' ] = getenv('DEFAULT_SECURITY_LEVEL') ?: 'impossible';

Default locale
Default locale for the help page shown with each session.
The default is 'en'. You may wish to set this to either 'en' or 'zh'.
_DVWA[ 'default_locale' ] = getenv('DEFAULT_LOCALE') ?: 'en';

Disable authentication
```

Figure 6.1: capture de la creation de la data base

Puis Depuis Kali, On ouvre on essaie d'accéder en passant par http pour inspecter le formulaire HTML. Le DVWA est intentionnellement de mauvaise qualité pour permettre d'avoir un environnement propice au test

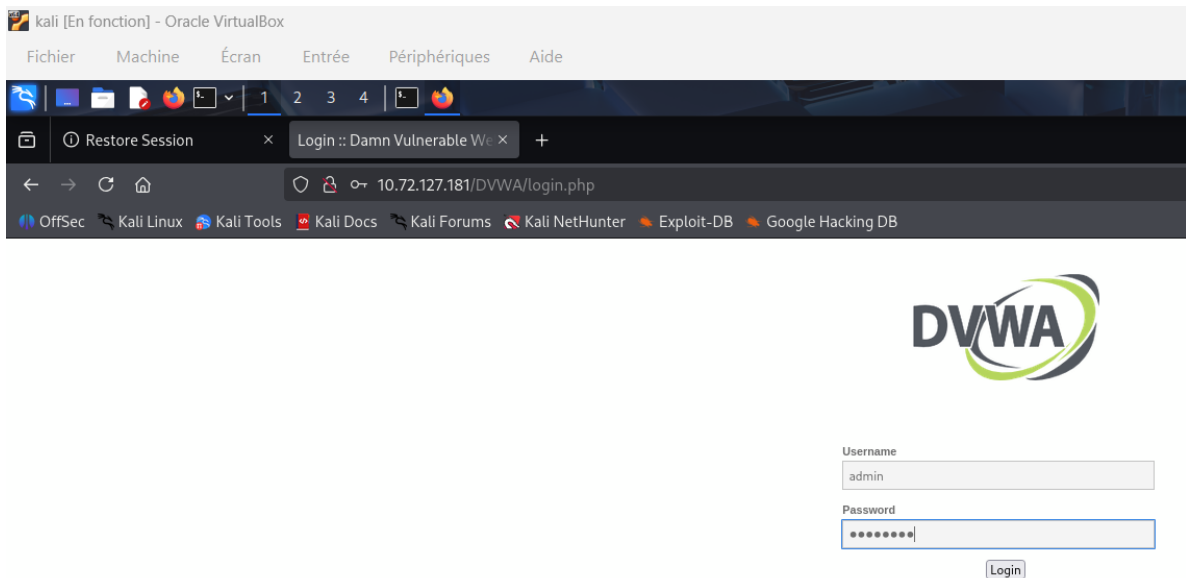


Figure 6.2: affichage du formulaire

6.1.2 Accès au formulaire

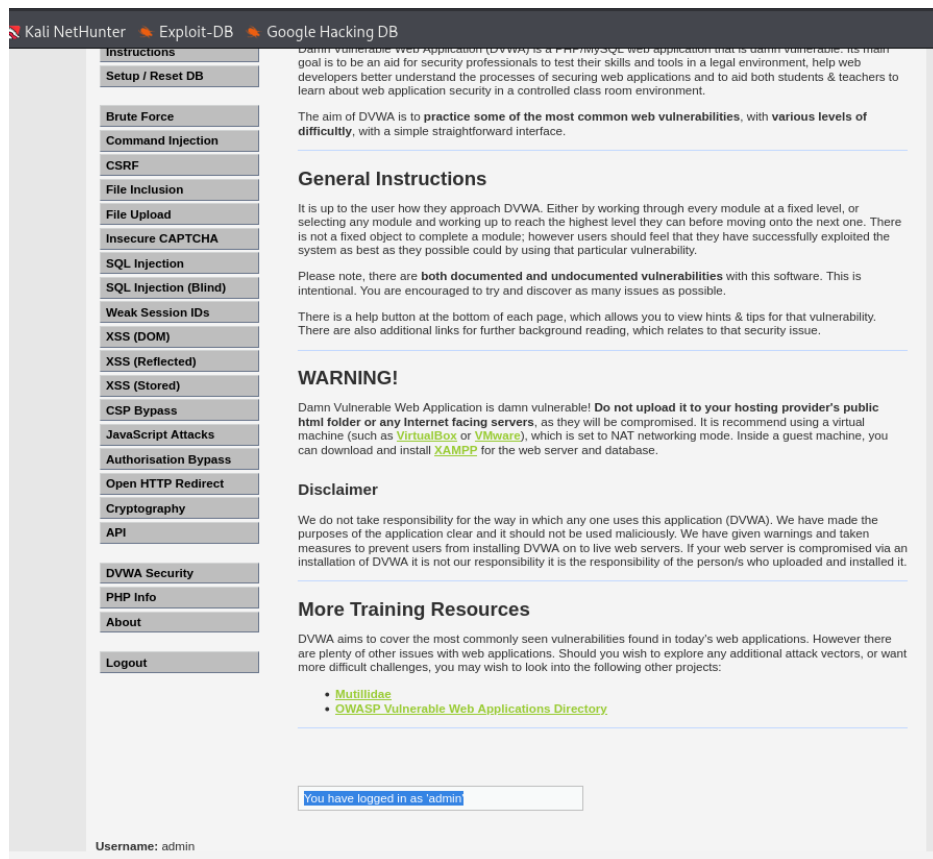


Figure 6.3: accès au formulaire :REUSSI

Chapter 7

Déploiement d'UFW — Contre-mesures

7.1 Introduction à UFW

7.1.1 Architecture interne d'UFW

UFW est un frontal à `iptables`. Quand on exécute `sudo ufw enable`, UFW génère automatiquement un ensemble de chaînes `iptables` (`ufw-before-input`, `ufw-user-input`, `ufw-after-input`, etc.) et les injecte dans les tables `filter` d'`iptables`.

```
1 sudo iptables -L -n -v
```

Listing 7.1: Visualisation des règles `iptables` générées par UFW

```
abdoulgafarouzoungana@Ubuntu:~$ sudo iptables -L -n -v
[sudo] password for abdoulgafarouzoungana:
Chain INPUT (policy ACCEPT 259K packets, 28M bytes)
pkts bytes target      prot opt in     out     source                destination
283K 44M ufw-before-logging-input 0 -- * * 0.0.0.0/0 0.0.0.0/0
283K 44M ufw-before-input 0 -- * * 0.0.0.0/0 0.0.0.0/0
265K 30M ufw-after-input 0 -- * * 0.0.0.0/0 0.0.0.0/0
260K 30M ufw-after-logging-input 0 -- * * 0.0.0.0/0 0.0.0.0/0
260K 30M ufw-reject-input 0 -- * * 0.0.0.0/0 0.0.0.0/0
260K 30M ufw-track-input 0 -- * * 0.0.0.0/0 0.0.0.0/0

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source                destination
0 0 ufw-before-logging-forward 0 -- * * 0.0.0.0/0 0.0.0.0/0
0 0 ufw-before-forward 0 -- * * 0.0.0.0/0 0.0.0.0/0
0 0 ufw-after-forward 0 -- * * 0.0.0.0/0 0.0.0.0/0
0 0 ufw-after-logging-forward 0 -- * * 0.0.0.0/0 0.0.0.0/0
0 0 ufw-reject-forward 0 -- * * 0.0.0.0/0 0.0.0.0/0
0 0 ufw-track-forward 0 -- * * 0.0.0.0/0 0.0.0.0/0

Chain OUTPUT (policy ACCEPT 135K packets, 5715K bytes)
pkts bytes target      prot opt in     out     source                destination
142K 6119K ufw-before-logging-output 0 -- * * 0.0.0.0/0 0.0.0.0/0
142K 6119K ufw-before-output 0 -- * * 0.0.0.0/0 0.0.0.0/0
136K 5759K ufw-after-output 0 -- * * 0.0.0.0/0 0.0.0.0/0
136K 5759K ufw-after-logging-output 0 -- * * 0.0.0.0/0 0.0.0.0/0
136K 5759K ufw-reject-output 0 -- * * 0.0.0.0/0 0.0.0.0/0
136K 5759K ufw-track-output 0 -- * * 0.0.0.0/0 0.0.0.0/0

Chain ufw-after-forward (1 references)
pkts bytes target      prot opt in     out     source                destination

Chain ufw-after-input (1 references)
pkts bytes target      prot opt in     out     source                destination
```

Figure 7.1: Visualisation des règles `iptables` générées par UFW

7.1.2 Commandes fondamentales

Table 7.1: Récapitulatif des commandes UFW utilisées dans ce projet

Commande	Signification
<code>sudo ufw enable</code>	Active le pare-feu (appliqué immédiatement et au démarrage)
<code>sudo ufw disable</code>	Désactive le pare-feu
<code>sudo ufw status</code>	Affiche l'état et les règles actives
<code>sudo ufw status numbered</code>	Affiche les règles avec leur numéro (utile pour insert/delete)
<code>sudo ufw status verbose</code>	Affiche le statut détaillé (politique par défaut, logging)
<code>sudo ufw allow 22/tcp</code>	Autorise tout trafic TCP entrant sur le port 22
<code>sudo ufw deny 8080/tcp</code>	Bloque tout trafic TCP entrant sur le port 8080
<code>sudo ufw limit 22/tcp</code>	Autorise SSH mais limite à 6 connexions/30s par IP
<code>sudo ufw deny from 10.72.177.50</code>	Bloque tout trafic provenant d'une IP spécifique
<code>sudo ufw insert 1 deny from ...</code>	Insère une règle en première position
<code>sudo ufw delete 3</code>	Supprime la règle numéro 3
<code>sudo ufw reset</code>	Réinitialise toutes les règles à zéro
<code>sudo ufw logging medium</code>	Active la journalisation à un niveau intermédiaire
<code>sudo ufw default deny incoming</code>	Règle par défaut : bloquer tout le trafic entrant
<code>sudo ufw default allow outgoing</code>	Règle par défaut : autoriser tout le trafic sortant

7.2 Protection contre l'attaque SSH (force brute)

7.2.1 Activation d'UFW et règle limit

La commande `sudo ufw limit 22/tcp` est particulièrement adaptée pour contrer les attaques par force brute. Elle utilise le module de suivi de connexion d'iptables pour compter le nombre de nouvelles connexions par IP :

```
1 sudo ufw enable
2 sudo ufw limit 22/tcp
3 sudo ufw status
```

Listing 7.2: Activation d'UFW avec limitation SSH — Machine Ubuntu

Information

Comportement de la règle limit : Toute adresse IP qui tente plus de **6 connexions en 30 secondes** sur le port 22 est automatiquement bloquée. Les connexions légitimes (une seule connexion, peu fréquente) ne sont pas affectées. Cette règle utilise les modules `recent` et `hashlimit` d'iptables en arrière-plan.

```
abdoulgafarouzoungana@Ubuntu:~$ sudo ufw limit 22/tcp
Rules updated
Rules updated (v6)
```

Figure 7.2: 22/tcp LIMIT IN Anywhere

7.2.2 Relance de l'attaque Hydra après protection

Nous relançons exactement la même commande Hydra depuis Kali :

```
1 hydra -l abdoulgafarouzoungana -P mini_dico.txt \
2 ssh://10.72.153.194 -t 4 -V
```

Listing 7.3: Tentative d'attaque SSH après activation de la règle limit

```
[RE-ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "xcvb" - 24 of 40 [child 3] (0/0)
[RE-ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "xcvb" - 24 of 40 [child 3] (0/0)
[ERROR] all children were disabled due too many connection errors
0 of 1 target completed, 0 valid password found
[INFO] Writing restore file because 2 server scans could not be completed
[ERROR] 1 target was disabled because of too many errors
[ERROR] 1 targets did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-03 18:02:27
```

```
(abdoulgafarouzoungana@PEXWELLDER)-[~]
$
```

Figure 7.3: Sortie de Hydra montrant les erreurs

A retenir

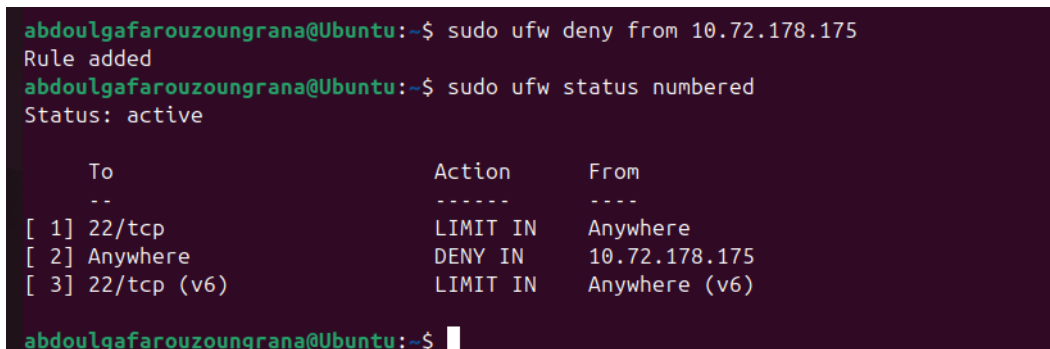
La règle `limit` est efficace. Après les premières tentatives, les connexions suivantes de 10.72.177.50 sont silencieusement ignorées (DROP) par UFW, causant un timeout côté Hydra. L'attaquant ne reçoit aucun message d'erreur explicite, ce qui rend le débogage de son attaque plus difficile.

7.2.3 Bannissement manuel de l'IP attaquante

En complément de la limitation de débit, nous ajoutons une règle de blocage explicite de l'IP de l'attaquant :

```
1 sudo ufw deny from 10.72.177.50
2 sudo ufw status numbered
```

Listing 7.4: Blocage explicite de l'IP de l'attaquant



```
abdoulgafarouzoungana@Ubuntu:~$ sudo ufw deny from 10.72.178.175
Rule added
abdoulgafarouzoungana@Ubuntu:~$ sudo ufw status numbered
Status: active

      To      Action      From
      --      -
[ 1] 22/tcp    LIMIT IN    Anywhere
[ 2] Anywhere  DENY IN     10.72.178.175
[ 3] 22/tcp (v6) LIMIT IN    Anywhere (v6)

abdoulgafarouzoungana@Ubuntu:~$
```

Figure 7.4: Terminal Ubuntu montrant `sudo ufw status` avec les deux règles : Anywhere DENY FROM 10.72.178.175 et 22/tcp LIMIT IN Anywhere

7.3 Protection contre l'exposition HTTP Python (port 8080)

7.3.1 Application de la règle deny

```
1 sudo ufw deny 8080/tcp
2 sudo ufw status
```

Listing 7.5: Blocage du port 8080 — Machine Ubuntu

```
abdoulgafarouzoungana@Ubuntu:~$ sudo ufw status numbered
Status: active

      To Action From
      --
[ 1] 22/tcp LIMIT IN Anywhere
[ 2] 8080 DENY IN Anywhere
[ 3] 22/tcp (v6) LIMIT IN Anywhere (v6)
[ 4] 8080 (v6) DENY IN Anywhere (v6)

abdoulgafarouzoungana@Ubuntu:~$
```

Figure 7.5: Terminal Ubuntu : `sudo ufw status` montrant la règle 8080/tcp DENY IN Anywhere

7.3.2 Vérification de l'efficacité

Depuis Kali, nous retentons l'accès :

```
(abdoulgafarouzoungana@PEXWELLDER)-[~]
$ wget http://10.72.152.67:8080/ex3
--2026-04-03 18:25:01-- http://10.72.152.67:8080/ex3
Connexion à 10.72.152.67:8080... échec : Connexion refusée.
```

Figure 7.6: Terminal Kali : CONNEXION REFUSÉ

A retenir

Le serveur Python tourne toujours sur Ubuntu, mais UFW empêche les paquets TCP à destination du port 8080 d'atteindre le processus.

7.4 Protection contre les attaques web (DVWA, port 80)

La limitation de débit s'applique également au port HTTP :

```
1 sudo ufw limit 80/tcp
2 sudo ufw status
```

Listing 7.6: Limitation du trafic HTTP pour protéger DVWA


```
abdoulgafarouzougrana@Ubuntu:~$ sudo ufw status numbered
Status: active

      To      Action      From
      --      -
[ 1] 22/tcp    LIMIT IN    Anywhere
[ 2] 8080      DENY IN    Anywhere
[ 3] 80        LIMIT IN    Anywhere
[ 4] 22/tcp (v6) LIMIT IN    Anywhere (v6)
[ 5] 8080 (v6) DENY IN    Anywhere (v6)
[ 6] 80 (v6)   LIMIT IN    Anywhere (v6)

abdoulgafarouzougrana@Ubuntu:~$
```

Figure 7.7: Terminal Ubuntu : `sudo ufw status` montrant toutes les règles actives incluant `80/tcp LIMIT IN Anywhere`

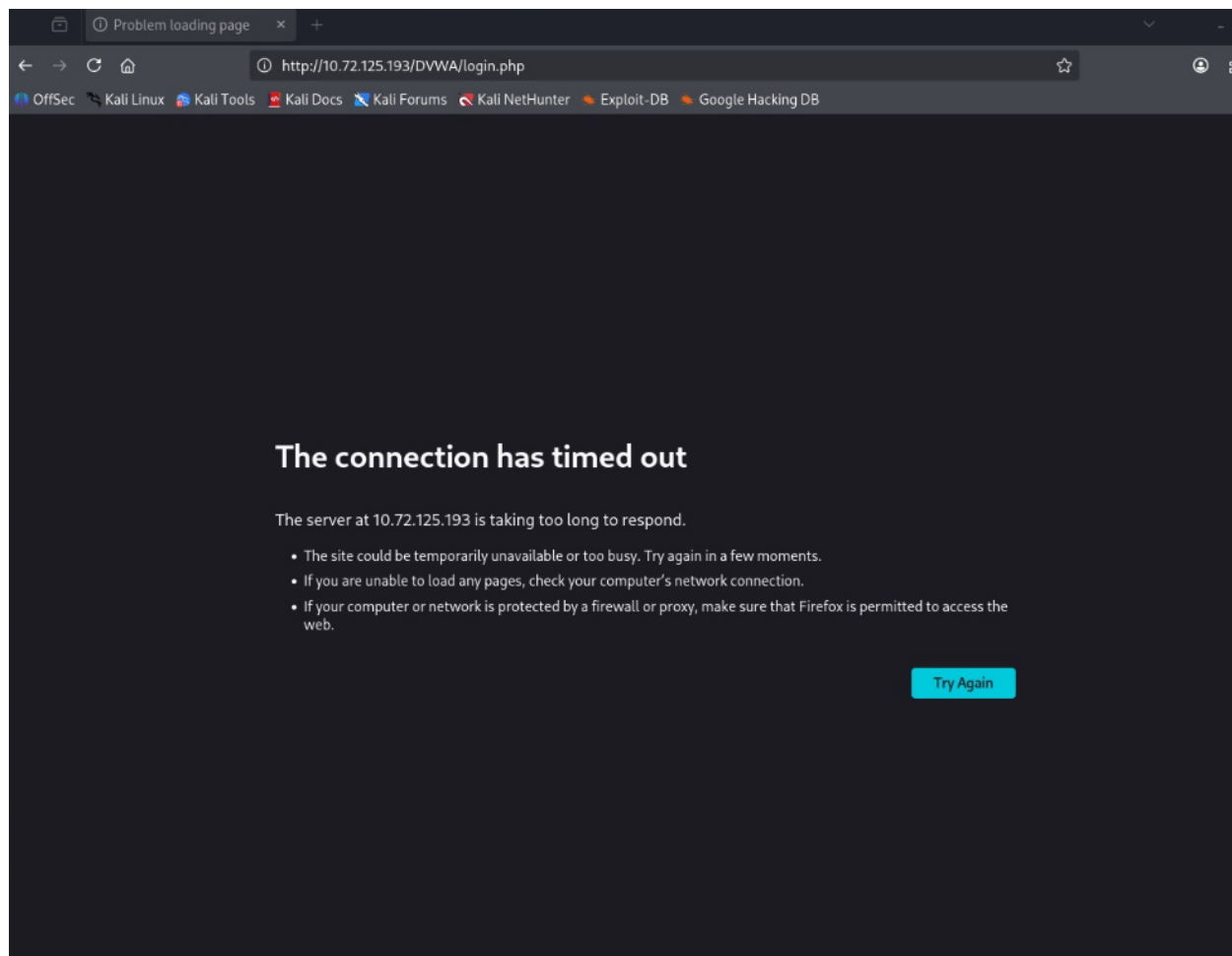


Figure 7.8: accès au DVWA bloqué

Information

La règle `limit 80/tcp` ralentit considérablement les attaques par dictionnaire sur formulaire HTTP. Hydra dépasse rapidement la limite de 6 connexions/30s et ses requêtes commencent à être bloquées. La protection est renforcée mais pas absolue : un attaquant patient qui étale ses requêtes dans le temps peut contourner cette mesure. C'est pourquoi la combinaison UFW + CAPTCHA + verrouillage de compte est recommandée en production.

Chapter 8

Analyse critique de l'ordre des règles UFW

8.1 Le principe “First Match Wins”

Definition

UFW (comme iptables) évalue les règles **dans l'ordre numérique croissant**. Dès qu'un paquet correspond à une règle (que ce soit `allow`, `deny` ou `limit`), cette règle est appliquée et l'évaluation s'arrête. Les règles suivantes ne sont jamais consultées pour ce paquet. Ce comportement est appelé **“First Match Wins”**.

Ce principe a des implications directes et parfois contre-intuitives sur l'efficacité du filtrage.

8.2 Scénario problématique : blocage inopérant

Imaginons la configuration suivante :

```
1 sudo ufw enable
2 sudo ufw allow 22/tcp                # Regle 1 : autoriser
   SSH
3 sudo ufw deny from 10.72.177.50      # Regle 2 : bloquer l'
   attaquant
```

Listing 8.1: Configuration problématique : order incorrect des règles

Visualisons l'état :

```
1 sudo ufw status numbered
2 # [1] 22/tcp                ALLOW IN    Anywhere
3 # [2] Anywhere              DENY IN    10.72.177.50
```

Listing 8.2: Affichage numéroté des règles dans l'ordre incorrect

Problème : Lorsqu'un paquet arrive depuis 10.72.177.50 à destination du port 22, UFW évalue les règles dans l'ordre :

1. Règle 1 : 22/tcp ALLOW — Le paquet correspond ! Il est immédiatement autorisé. L'évaluation s'arrête.
2. Règle 2 : jamais atteinte pour ce paquet.

Relançons Hydra pour vérifier : l'attaque réussit à nouveau malgré la règle de blocage.

```
abdoulgafarouzoungana@Ubuntu:~$ sudo ufw status numbered
Status: active

      To Action From
      --
[ 1] 22/tcp ALLOW IN Anywhere
[ 2] Anywhere DENY IN 10.72.178.175
[ 3] 22/tcp (v6) ALLOW IN Anywhere (v6)

abdoulgafarouzoungana@Ubuntu:~$
```

Figure 8.1: règle dans le mauvais ordre

```
(abdoulgafarouzoungana@PEXWELLDER)~$
$ hydra -l abdoulgafarouzoungana -P mini_dico.txt \
ssh://10.72.153.175 -t 4 -V
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizati
ons, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-03 18:48:32
[DATA] max 4 tasks per 1 server, overall 4 tasks, 8 login tries (l:1/p:8), ~2 tries per task
[DATA] attacking ssh://10.72.153.175:22/
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "password" - 1 of 8 [child 0] (0/0)
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "admin" - 2 of 8 [child 1] (0/0)
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "password" - 3 of 8 [child 2] (0/0)
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "123456" - 4 of 8 [child 3] (0/0)
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "45666" - 5 of 8 [child 2] (0/0)
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "5446313" - 6 of 8 [child 0] (0/0)
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "65xcvbn" - 7 of 8 [child 1] (0/0)
[ATTEMPT] target 10.72.153.175 - login "abdoulgafarouzoungana" - pass "1234" - 8 of 8 [child 3] (0/0)
[22][ssh] host: 10.72.153.175 login: abdoulgafarouzoungana password: 1234
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-03 18:48:38
(abdoulgafarouzoungana@PEXWELLDER)~$
```

Figure 8.2: L'attaque fonctionne malgré le blocage de l'adresse IP

8.3 Correction : Insérer le blocage en première position

Pour corriger ce problème, nous devons placer la règle de blocage **avant** la règle d'autorisation :

```

abdoulgafarouzougrana@Ubuntu:~$ sudo ufw status numbered
Status: active

      To Action      From
      -- -
[ 1] Anywhere DENY IN 10.72.178.175
[ 2] 22/tcp   ALLOW IN Anywhere
[ 3] 22/tcp (v6) ALLOW IN Anywhere (v6)

abdoulgafarouzougrana@Ubuntu:~$

```

Figure 8.3: regle dans le bon ordre

```

(abdoulgafarouzougrana@PEXWELLDER)~$ hydra -l abdoulgafarouzougrana -P mini_dico.txt \
ssh://10.72.153.175 -t 4 -V
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizati
ons, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-03 18:53:54
[DATA] max 4 tasks per 1 server, overall 4 tasks, 8 login tries (l:1/p:8), ~2 tries per task
[DATA] attacking ssh://10.72.153.175:22/
[ERROR] could not connect to ssh://10.72.153.175:22 - Timeout connecting to 10.72.153.175

```

Figure 8.4: L'attaque ne fonctionne plus

Maintenant, le flux de décision est correct :

1. Règle 1 : DENY FROM 10.72.177.50 — Le paquet de l'attaquant correspond. Il est bloqué. L'évaluation s'arrête.
2. Règle 2 : 22/tcp ALLOW — jamais atteinte pour les paquets de l'attaquant.

A retenir

Règle d'or : Dans un pare-feu, les règles de blocage spécifiques doivent toujours être placées **avant** les règles d'autorisation générales. L'ordre est aussi important que le contenu des règles.

8.4 Tableau récapitulatif de l'impact de l'ordre

Table 8.1: Impact de l'ordre des règles sur le filtrage

Ordre	Configuration	Résultat pour l'attaquant
Incorrect	[1] allow 22/tcp, [2] deny from IP	Autorisé (règle 1 match en premier)
Correct	[1] deny from IP, [2] allow 22/tcp	Bloqué (règle 1 match en premier)

Chapter 9

Politique de sécurité finale

9.1 Principes directeurs

La politique de sécurité finale repose sur trois principes fondamentaux :

1. **Principe du moindre privilège** : Tout ce qui n'est pas explicitement autorisé est interdit (*default deny*).
2. **Défense en profondeur** : Plusieurs couches de protection (limite de taux + blocage IP + journalisation).
3. **Traçabilité** : Toutes les décisions du pare-feu sont enregistrées pour permettre un audit post-incident.

9.2 Configuration complète et commentée

```

abdoulgafarouzungrana@Ubuntu:~$ # Etape 1 : Réinitialisation propre de toutes les règles existantes
sudo ufw reset

# Etape 2 : Politique par défaut - bloquer tout le trafic entrant
#
# autoriser tout le trafic sortant
sudo ufw default deny incoming
sudo ufw default allow outgoing

# Etape 3 : Autoriser SSH avec limitation anti-brute-force
#
# (max 6 tentatives par 30 secondes par IP)
sudo ufw limit 22/tcp

# Etape 4 : Autoriser le trafic web standard
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

# Etape 5 : Bloquer explicitement le port 8080
#
# (serveur de développement, ne jamais exposer en production)
sudo ufw deny 8080/tcp

# Etape 6 : Activer la journalisation niveau medium
#
# (enregistre les paquets bloqués et certains paquets autorisés)
sudo ufw logging medium

# Etape 7 : Activer le pare-feu
sudo ufw enable

# Etape 8 : Vérification du statut final
sudo ufw status verbose
Resetting all rules to installed defaults. Proceed with operation (y|n)? y
Backing up 'user.rules' to '/etc/ufw/user.rules.20260403_183808'
Backing up 'before.rules' to '/etc/ufw/before.rules.20260403_183808'
Backing up 'after.rules' to '/etc/ufw/after.rules.20260403_183808'
Backing up 'user6.rules' to '/etc/ufw/user6.rules.20260403_183808'
Backing up 'before6.rules' to '/etc/ufw/before6.rules.20260403_183808'
Backing up 'after6.rules' to '/etc/ufw/after6.rules.20260403_183808'

```

Figure 9.1: 1

```

Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
Rules updated
Rules updated (v6)
Rules updated
Rules updated (v6)
Rules updated
Rules updated (v6)
Rules updated
Rules updated (v6)
Logging enabled
Firewall is active and enabled on system startup
Status: active
Logging: on (medium)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp LIMIT IN Anywhere
80/tcp ALLOW IN Anywhere
443/tcp ALLOW IN Anywhere
8080/tcp DENY IN Anywhere
22/tcp (v6) LIMIT IN Anywhere (v6)
80/tcp (v6) ALLOW IN Anywhere (v6)
443/tcp (v6) ALLOW IN Anywhere (v6)
8080/tcp (v6) DENY IN Anywhere (v6)
abdoulgafarouzungrana@Ubuntu:~$

```

Figure 9.2: affichage

9.3 Récapitulatif de la politique

Table 9.1: Politique de sécurité finale UFW

Service / Port	Règle	Justification
Tout le trafic entrant	DENY (défaut)	Principe de moindre privilège
Tout le trafic sortant	ALLOW (défaut)	La machine doit pouvoir initier des connexions
22/TCP (SSH)	LIMIT	Accès administratif nécessaire, mais protégé contre la force brute
80/TCP (HTTP)	ALLOW	Service web légitime
443/TCP (HTTPS)	ALLOW	Service web sécurisé
8080/TCP	DENY	Port de développement à ne jamais exposer
Logging	medium	Traçabilité des événements de sécurité

9.4 Surveillance continue des journaux UFW

La politique de sécurité ne s'arrête pas à la configuration des règles. Une surveillance active est indispensable :

```

abdoulgafarouzuougrana@Ubuntu:~$ sudo tail -f /var/log/uFW.log | grep BLOCK
2026-04-03T18:54:56.042444+00:00 localhost kernel: [UFW BLOCK] IN=enp0s3 OUT= MAC=08:00:27:b9:75:73:88:51:fb:d7:6a:00:00:00 SRC=10.8.31.8 DST=10.72.153.175 LEN=52 TOS=0x00 PREC=0x00 TT
L=127 ID=49260 DF PROTO=TCP SPT=64362 DPT=7680 WINDOW=65535 RES=0x00 SYN URG=0
^C
abdoulgafarouzuougrana@Ubuntu:~$ sudo grep "Failed password" /var/log/auth.log | awk '{print $11}' | sort | uniq -c | sort -rn | head -10
grep: /var/log/auth.log: binary file matches
 12 53922
  6 60988
  6 60978
  6 60364
  6 60350
  6 60334
  6 60324
  6 59902
  6 59880
  6 59716
abdoulgafarouzuougrana@Ubuntu:~$ sudo grep "Failed password" /var/log/auth.log | awk '{print $11}' | sort | uniq -c | sort -rn | head -10
grep: /var/log/auth.log: binary file matches
 12 53922
  6 60988
  6 60978
  6 60364
  6 60350
  6 60334
  6 60324
  6 59902
  6 59880
  6 59716

```

Figure 9.3: Surveillance des journaux UFW en temps réel

Information

Aller plus loin : En production, UFW est généralement associé à **fail2ban**, un outil qui analyse automatiquement les journaux et ajoute dynamiquement des règles de blocage pour les IPs identifiées comme malveillantes. Cette combinaison permet un bannissement automatique sans intervention humaine.

Chapter 10

Conclusion

10.1 Bilan du projet

Ce projet a permis d’atteindre l’ensemble des objectifs fixés. En adoptant une approche “attaque avant défense”, nous avons pu comprendre en profondeur non pas seulement *comment* configurer UFW, mais *pourquoi* chaque règle est nécessaire.

Les principaux enseignements sont les suivants :

1. **Un service non protégé est un service compromis.** Les trois attaques ont réussi en quelques secondes dans l’environnement non protégé. La surface d’attaque d’un serveur sans pare-feu est extrêmement grande.
2. **La commande `limit` est un premier rempart puissant.** En limitant le débit de connexions, elle rend les attaques par force brute impraticables sans éliminer l’accès légitime.
3. **L’ordre des règles est aussi important que leur contenu.** Une règle de blocage placée après une règle d’autorisation plus générale est inopérante.
4. **La journalisation est indispensable.** Sans logs, une attaque peut passer complètement inaperçue.
5. **Le principe de moindre privilège s’applique aux ports.** Tout port non nécessaire doit être fermé par défaut. La politique `default deny incoming` est la bonne pratique universelle.

10.2 Perspectives

Ce projet constitue une base solide que l’on peut enrichir avec :

- L'intégration de **fail2ban** pour le bannissement automatique.
- L'utilisation de l'authentification **par clés SSH** à la place des mots de passe (rend la force brute inutile).
- La mise en place d'un **IDS/IPS** (Snort, Suricata) pour une détection plus fine.
- L'exploration de **nftables**, le successeur moderne d'iptables sur les kernels récents.

Appendix A

Glossaire

UFW	Uncomplicated Firewall — Interface simplifiée à iptables sous Linux.
iptables	Outil de bas niveau pour la configuration du pare-feu Linux.
Nmap	Network Mapper — Outil de reconnaissance réseau.
Hydra	Outil de force brute réseau multi-protocoles.
DVWA	Damn Vulnerable Web Application — Application web vulnérable à des fins pédagogiques.
SSH	Secure Shell — Protocole d'administration distante chiffrée.
Force brute	Technique d'attaque consistant à essayer systématiquement des mots de passe.
CVE	Common Vulnerabilities and Exposures — Répertoire des vulnérabilités connues.
IDS/IPS	Intrusion Detection / Prevention System — Système de détection ou prévention d'intrusions.
DROP	Action iptables qui écarte silencieusement les paquets (sans réponse à l'expéditeur).
REJECT	Action iptables qui bloque les paquets en envoyant une réponse d'erreur à l'expéditeur.

MOT DE LA FIN

*« Nous tenons à exprimer notre profonde gratitude à
Madame Asmae EL KASSIRI pour nous avoir
permis d’appréhender les réalités de la sécurité réseau
à travers ce projet.*

*Cette immersion nous a prouvé qu’un système
initialement vulnérable peut devenir un environnement
résistant grâce à une configuration rigoureuse d’UFW.
En passant de l’attaque à la remédiation, nous avons
compris que la traçabilité et le filtrage constituent le
rempart fondamental de tout système d’information. »*

“Mieux vaut un pare-feu bien configuré qu’une brèche silencieuse.”

L’Équipe du Projet N°9

ZOUNGRANA Abdoul
Gafarou

ABOUBAKAR
AbdelAziz

DIN Isaac Kaougahi

KAFANDO Mohamed